

Open in app ↗



Medium



Search



Write

Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)

AI vs Real Photo detection



IrmakOrhan

Follow

8 min read · May 20, 2025



Alphoto Detector - a Hugging Face Space by irmakkorhan

Discover amazing ML apps made by the community

huggingface.co

You can easily test models performance by gradio

The recent advancements in artificial intelligence are nothing short of fascinating. It's hard to find a field untouched by AI today, and image generation is one of the most impressive examples. AI-generated visuals have reached a point where they're often indistinguishable from real photographs. You can see this for yourself on websites like This Person Does Not Exist where every refresh generates a new, entirely fake yet photorealistic human face.

Inspired by this, I decided to build a deep learning model that can detect whether an image is AI-generated or a real photograph. It is a very up-to-date application that can be used in areas such as deepfake detection, visual security, and fake product recognition in the real world.

Why I used deep learning ?

I chose to deep learning, a kind of machine learning that uses artificial neural networks to simulate the structure and operation of the human brain, because the task involves identifying minute patterns in images that are frequently invisible to the human eye. Because deep learning can automatically acquire hierarchical features straight from raw input, it performs exceptionally well in applications involving unstructured data, like text, audio, and images.

Unlike traditional picture classification methods that require manual feature engineering, deep learning allows the model to learn what to look for on its own, provided it has enough data and processing power. Because the difference between real and artificial intelligence (AI)-generated photos is often subtle and context-dependent, this makes it especially helpful in this regard.

Lets before diving into the project itself, I'd like to walk you through some essential concepts that form the foundation of this work.

A1. Terminology:

-Weights are the “power” values of the connections in the artificial neural network. When one neuron sends a signal to another, the weight determines how important this signal is.

-**Batching** is the process of dividing the training data into smaller pieces and processing them sequentially, rather than giving the entire training data to the model at once.

- **Epoch** means that the entire training data passes through the model once.

-**Transfer learning** is you take “a brain that has already learned a lot” → You change the last few layers “according to your task” and retrain it.

- **Kaiming** Initialization is a method that defines how the weights are initially determined in deep neural networks. The weights of the model are randomly initialized. FastAI & PyTorch already do automatic initialization with Kaiming for ReLU. When you call `cnn_learner(...)`: the model's weights are automatically initialized according to Kaiming.

- Model like **ResNet18**

- They are usually trained on huge datasets (1.2 million images like ImageNet).

- The first layers of these models have learned general visual features.

- You just adapt them to your task by changing the last layers (e.g. FAKE/REAL distinction).

- **Finder Algorithm** is a technique used to find the most appropriate learning rate to be used in training the model, manually or through visual analysis.

- Training starts with a very small learning rate (e.g. $1e-8$).

- As training continues for several batches, the learning rate is gradually increased.
- The loss value in each batch is recorded.
- When the learning rate exceeds a certain threshold, the loss increases suddenly (the model breaks down).
- The learning rate at the minimum loss point just before that point is considered “ideal”.

In FastAI, this process is done with a single line: `learn.lr_find()`

A2. Problems Encountered During the Project and Their Solutions

1. Problem : As I was nearing the end of my benchmark training, it reset and restarted the training (even though I didn't tell it to restart). It was frustrating to realize this was happening when I was at 90% of the training.

Reason : This is because Google Colab silently resets the runtime in the background. Colab silently resets your session (RAM, GPU, variables). But there is no change in the interface, the cells appear to be still. However, the variables are no longer in memory.

Solutions: Save your model as soon as training is completed (`learn.export('model.pkl')`). If the runtime resets or you reopen the notebook, you can call it back (`learn = load_learner('model.pkl')`)

2. Problem : The training time is so long

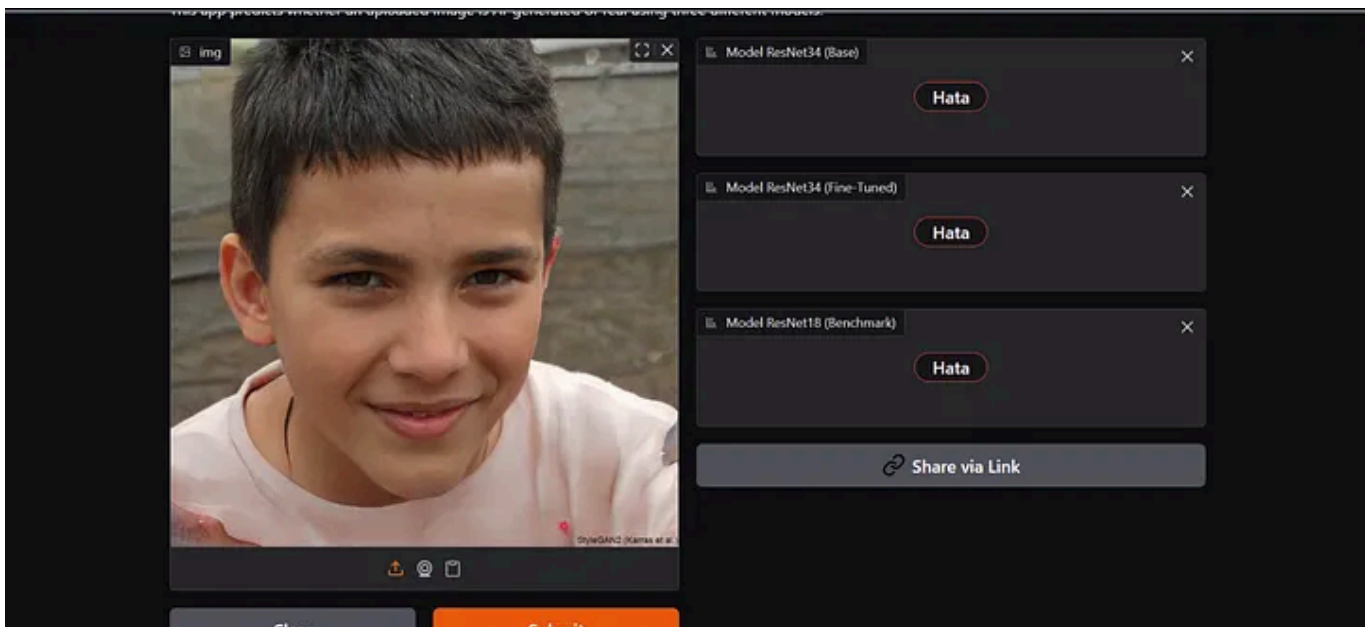
Reason : Using a CPU may lead to significantly longer training times and inefficiencies, especially as the dataset or model size grows.

Solution: when I switched to GPU, GPU became a much faster method because GPU can perform matrix operations in parallel and the input and output of the model are constantly multiplied as a matrix while being trained.

3. Problem: The model I trained with the first dataset I received did not give satisfied results

Solution : So to avoid overfitting and underfitting, I downloaded additional datasets from the internet so that I could train my model better.

4. Problem: I uploaded the photo during the gradio testing phase and received an error as follows.



Reason: We need to first convert the incoming image to the prediction function using fastai's `PILImage.create(img)` method.



Solution: `"img = PILImage.create(img)"` worked in my app.py class

Colab explanation

PartA : General Step

First I installed the necessary packages and download the data and create a datablock

```
[ ] from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[ ] !pip install -Uqq fastai
zip_path = "/content/drive/MyDrive/colab_cifake/archive.zip"

import zipfile
import os

extract_path = "/content/dataset"
os.makedirs(extract_path, exist_ok=True)

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Extraction complete. Files are now in:", extract_path)
```

```
[ ] train_path = Path("/content/dataset/train")

dblock = DataBlock(
    #A.2.1 Define the blocks:
    blocks=(ImageBlock, CategoryBlock),
    #A.2.2 - Define the means of getting data into DataBlock:
    get_items=get_image_files,
    splitter=RandomSplitter(seed=42),
    #A.2.3 - Define how to get the attributes
    get_y=parent_label,
    #A.2.4 - Define data transformations
    item_tfms=Resize(256),           # Presizing (item-level)
    batch_tfms=aug_transforms()      # Augmentations (batch-level)
)

# Create a DataLoaders object from the DataBlock, with a batch size of 32
dls = dblock.dataloaders(train_path, bs=32)
# Display 9 sample images from a single batch to visually verify labels and image transformations
#A.3.1 Show batch
# The labels (FAKE and REAL) are visually confirmed via show_batch()
#A.3.2 Check the labels
dls.show_batch(max_n=9, figsize=(6,6))
```

Part B : Train a simple model and check with confusion matrix

Before diving into advanced techniques, I first needed a benchmark. For that, I built a simple yet effective model using ResNet18, a lightweight convolutional neural network architecture that's often used as a starting point in image classification tasks.

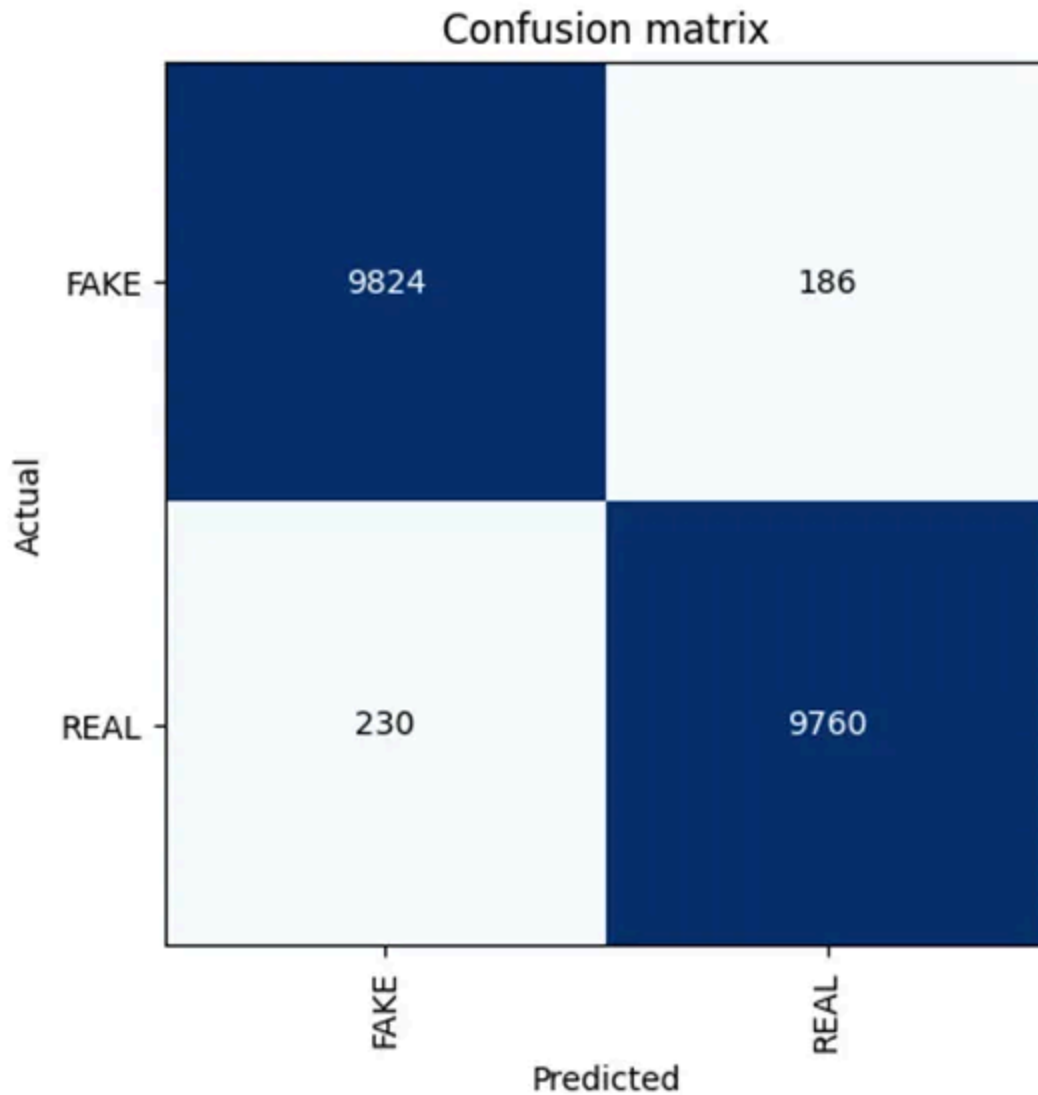
I initialized a convolutional neural network (CNN) using the fastai library's `vision_learner()` function. The model is tfine-tuned for 3 epochs.

- `fine_tune()` starts by freezing the early layers (which contain general image features like edges and colors).
- Then, it unfreezes the model and updates all layers using the new dataset.

Epoch 0 — Accuracy: ~92.2%

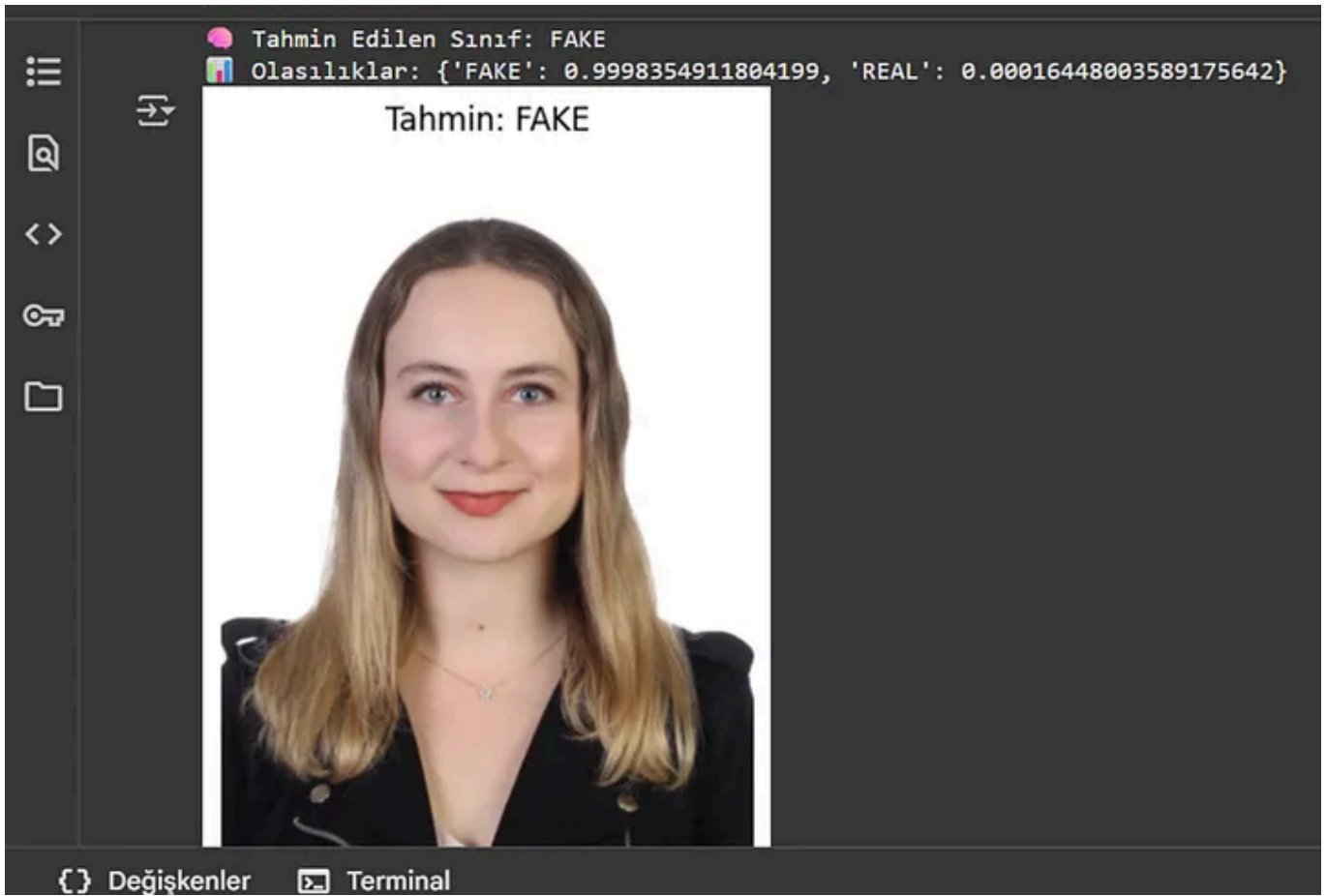
Epoch 1 — Accuracy: ~96.1%

Epoch 2 — Accuracy: ~97.5%



Then export the model by: `learn.export('resnet18-benchmark.pkl')`

Part B.2: Problematic side of the model: Since accuracy is not a very reliable matrix, I uploaded an image to test my model but it gave an incorrect prediction.



Part C : Obtained a new dataset by mixing old data with data I collected from duckduckgo and various sources on the internet.

```
[ ] !pip install duckduckgo-search

from duckduckgo_search import DDGS
import os, io, hashlib, requests, time
from PIL import Image

# label
classes = {
    "FAKE": [
        "stylegan face",
        "thispersondoesnotexist face",
        "AI generated portrait photo",
        "artificial intelligence human face",
        "midjourney face portrait"
    ],
    "REAL": [
        "passport photo of real person",
        "professional corporate headshot",
        "real human face photo",
        "linkedin profile photo",
        "school yearbook portrait"
    ]
}
```

```
[ ] from pathlib import Path
import shutil

# Dcorrect path
extracted_path = Path("/content/extracted_dataset_force/train")
target_path = Path("/content/drive/MyDrive/ai_vs_real_dataset")

# Copy FAKE and REAL folders to destination
for label in ['FAKE', 'REAL']:
    src = extracted_path / label
    dst = target_path / label

    if src.exists():
        print(f"📁 {label} kopyalanıyor...")
        for file in src.glob("*"):
            target_file = dst / file.name
            if not target_file.exists():
                shutil.copy(file, target_file)
    else:
        print(f"❌ {label} bulunamadı.")
```

Part D : Fine tune the model by new datasets and download the model

I trained my model for 4 epochs using the optimum base learn with the newly generated dataset and saved it. I trained my model using A100GPU.

```
[ ] learn = vision_learner(dls, resnet34, metrics=accuracy)
learn.fine_tune(epochs=4, base_lr=2e-3)
```

Downloading: "https://download.pytorch.org/models/resnet34-b627a593.pth" to /root/.cache/torch/hub/checkpoints/resnet34-b627a593.pth
100%|██████████| 83.3M/83.3M [00:00<00:00, 184MB/s]

epoch	train_loss	valid_loss	accuracy	time
0	0.254163	0.237608	0.903273	1:26:00

/usr/local/lib/python3.11/dist-packages/PIL/Image.py:1843: UserWarning: Palette images with Transparency expressed in bytes should be converted to RGB.
warnings.warn(
/usr/local/lib/python3.11/dist-packages/PIL/Image.py:1843: UserWarning: Palette images with Transparency expressed in bytes should be converted to RGB.
warnings.warn(
epoch train_loss valid_loss accuracy time
0 0.181393 0.113709 0.957730 02:32
1 0.112970 0.087164 0.969799 02:33
2 0.071784 0.060719 0.978806 02:34
3 0.043595 0.050586 0.981396 02:35
/usr/local/lib/python3.11/dist-packages/PIL/Image.py:1843: UserWarning: Palette images with Transparency expressed in bytes should be converted to RGB.
warnings.warn(
/usr/local/lib/python3.11/dist-packages/PIL/Image.py:1843: UserWarning: Palette images with Transparency expressed in bytes should be converted to RGB.

Part F : Advance technique

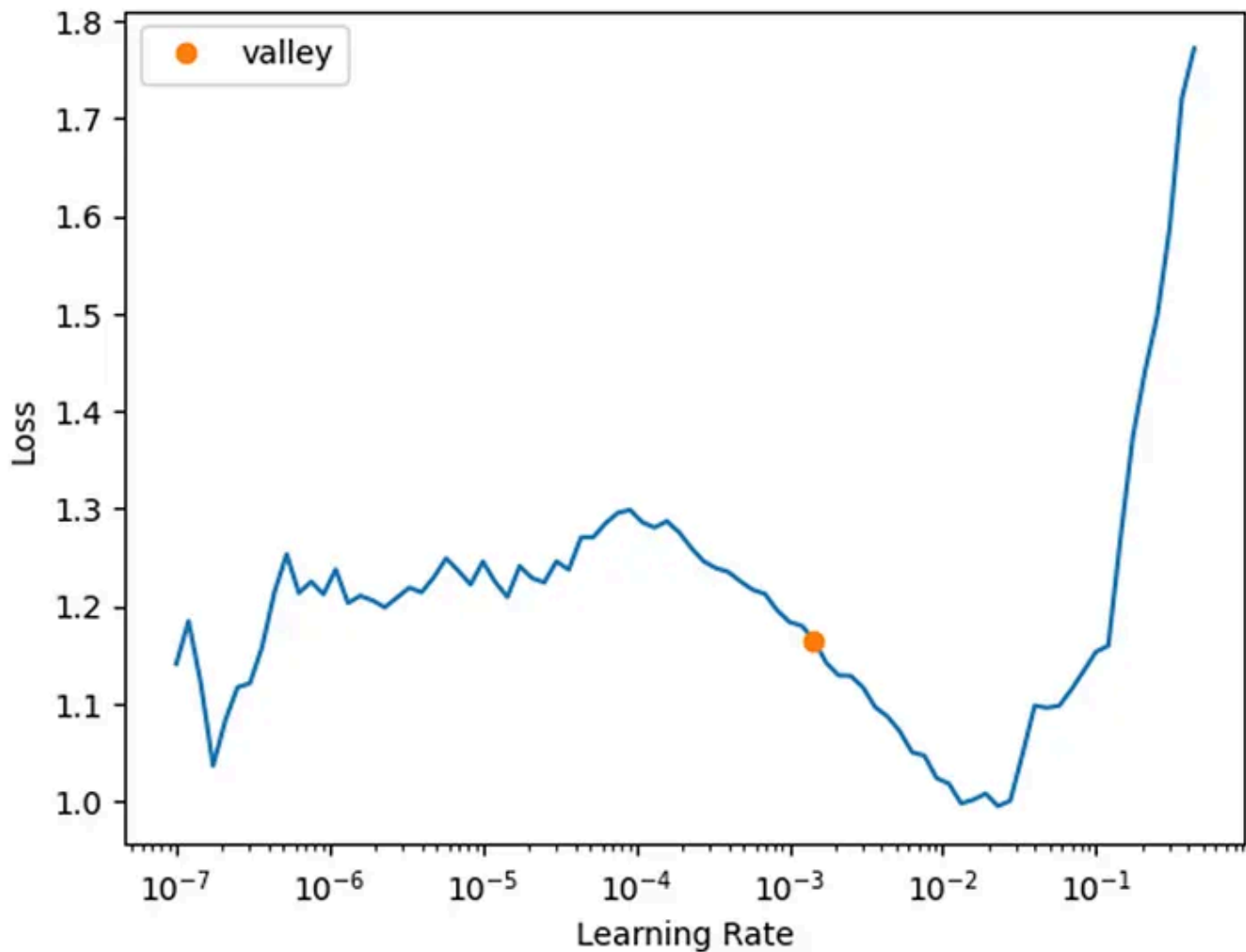
After building my baseline model using ResNet18 and achieving solid accuracy, I wanted to push things further. This meant diving into the deep learning toolbox and applying **advanced training techniques** — not just for performance, but also for understanding how subtle tweaks in training can reshape outcomes.

- **Learning Rate Finder:** a tool that helps you identify the most optimal learning rate before training even begins. Instead of guessing, I let the model try a range of learning rates and observe how the loss changed.



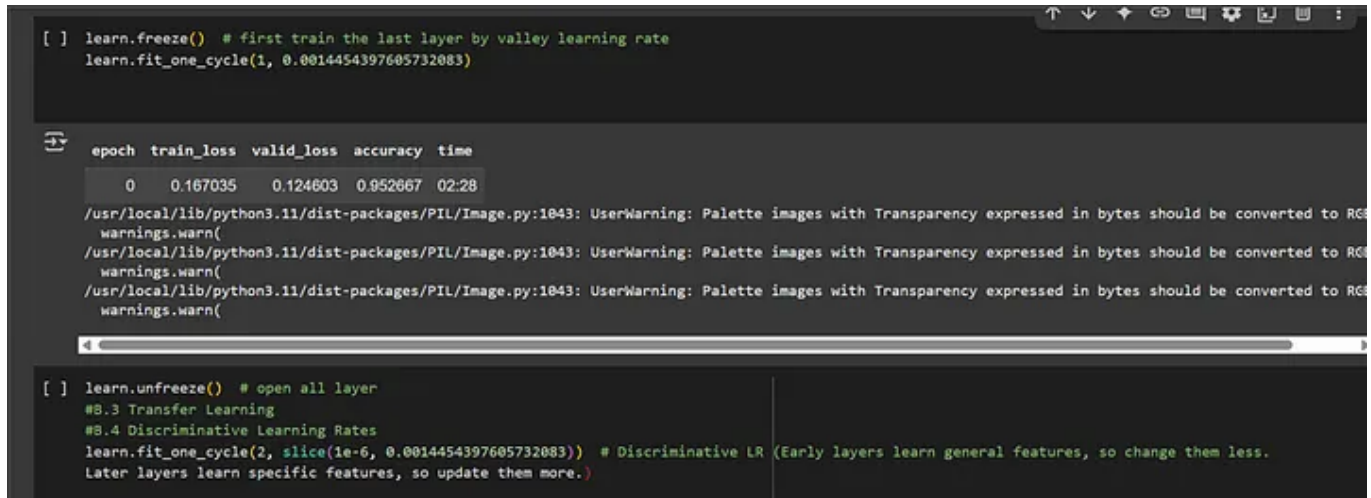
```
from fastai.vision.all import *
from fastai.callback.fp16 import *

learn = vision_learner(dls, resnet34, metrics=accuracy).to_fp16()
#B.1 Learning Rate Finder
lr_valley = learn.lr_find().valley
print(f"Valley LR: {lr_valley}")
```



- After setting up my model and selecting an optimal learning rate using the learning rate finder, I moved on to two powerful techniques that are often used in combination: **Transfer Learning** and **Discriminative Learning Rates**.

I froze all the pretrained layers of the model and **only trained the final classification layer**. letting the model retain general image features it has already learned from a large dataset (like ImageNet), while focusing on adapting to the new task at hand. Once the classifier was well-adjusted, I **unfroze the entire model** to let all layers adapt slightly to the new dataset. But here's the key: not all layers should be updated equally.



```
[ ] learn.freeze() # first train the last layer by valley learning rate
learn.fit_one_cycle(1, 0.0014454397605732083)
```

epoch	train_loss	valid_loss	accuracy	time
0	0.167035	0.124603	0.952667	02:28

```
/usr/local/lib/python3.11/dist-packages/PIL/Image.py:1043: UserWarning: Palette images with Transparency expressed in bytes should be converted to RGBA images
warnings.warn(
/usr/local/lib/python3.11/dist-packages/PIL/Image.py:1043: UserWarning: Palette images with Transparency expressed in bytes should be converted to RGBA images
warnings.warn(
/usr/local/lib/python3.11/dist-packages/PIL/Image.py:1043: UserWarning: Palette images with Transparency expressed in bytes should be converted to RGBA images
warnings.warn(

[ ] learn.unfreeze() # open all layer
#B.3 Transfer Learning
#B.4 Discriminative Learning Rates
learn.fit_one_cycle(2, slice(1e-6, 0.0014454397605732083)) # Discriminative LR (Early layers learn general features, so change them less.
Later layers learn specific features, so update them more.)
```

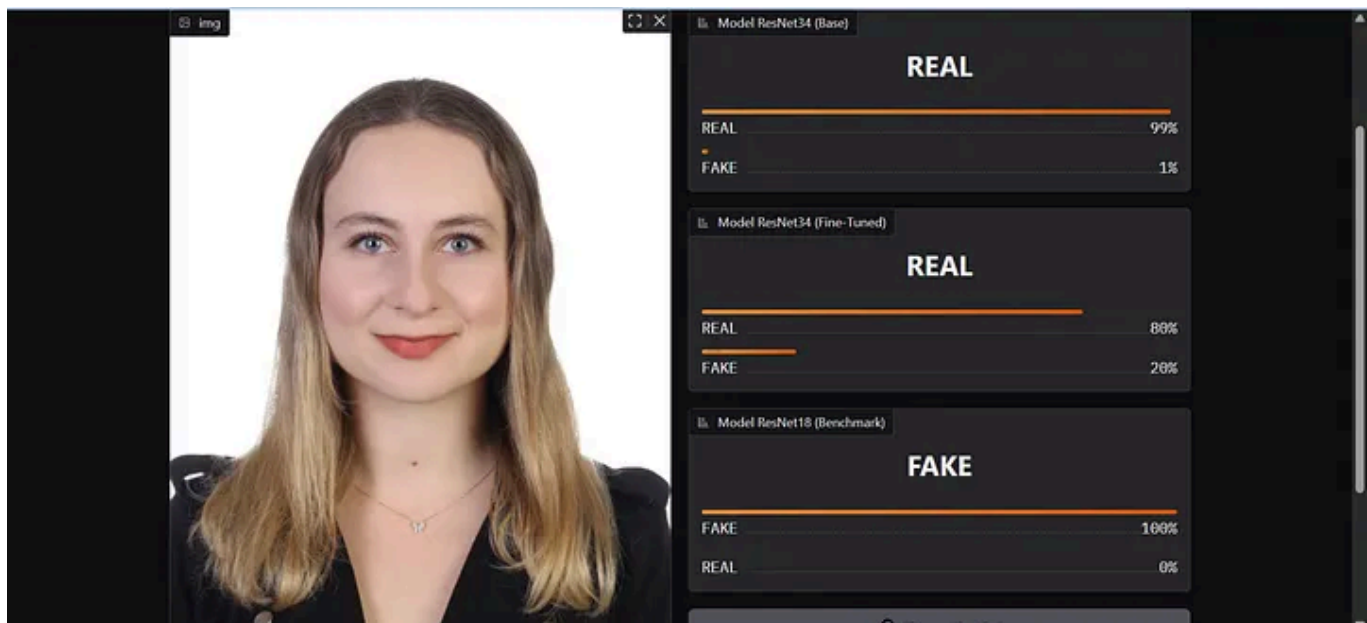
Instead of using a single learning rate for all layers, I applied **discriminative learning rates** using fastai's `slice()` function. The early layers have already **learned generic features** that are useful for most vision tasks, while later layers need to specialize for my task: detecting whether an image is **real or AI-generated**.

This combination of techniques — **freezing, then unfreezing**, paired with **layer-specific learning rates** — allowed the model to adapt smoothly without losing its pretrained power.

And finally, I also downloaded this model too.

In conclusion, I explored how working with different datasets and tweaking fine-tuning settings can significantly impact model performance. In my first model, I used a dataset directly from Kaggle. For the second one, I expanded the dataset and fine-tuned the model to improve accuracy. In the final model, I applied advanced techniques such as a learning rate finder to reach the most optimal results.

As we can observe, now I uploaded real picture and more advanced model which is trained by advanced technique and mixed dataset, predicted correctly.



As AI continues to evolve, we can expect even more realistic images to be generated. That's why detection models must also keep up with this rapid progress. Staying up-to-date and continuously improving these models with new methods is crucial in this ever-changing landscape.